INTELLIGENT API THREAT DETECTION USING GRAPH LEARNING

A PROJECT REPORT

Submitted by

KAVIYA J.J (2116231901019)

SASIKUMAR B. (2116231901047)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)





RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI

BONAFIDE CERTIFICATE

Certified that this Project report titled "**INTELLIGENT API THREAT DETECTION USING GRAPH LEARNING**" is the bonafide work of "**KAVIYA J J (2116231901019), SASIKUMAR B (2116231901047)**" who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. P. Tamilselvi, M.E., Ph.D.,

HEAD OF THE DEPARTMENT

PROFESSOR
Department of Computer Science and Engineering (Cyber Security)
Rajalakshmi Engineering College
Chennai – 602 105

SIGNATURE

Mrs. T Seetha Lakshmi, M.E (CSE),

SUPERVISOR

ASSISTANT PROFESSOR
Department of Computer Science and Engineering (Cyber Security)
Rajalakshmi Engineering College
Chennai – 602 105

Submitted to Project Viva-Voce Examination held on 07/10/2026

Internal Examiner

External Examiner

RAJALAKSHMI ENGINEERING COLLEGE

CHENNAI

ABSTRACT

Application Programming Interfaces (APIs) have become an important part of modern applications by enabling communication between different services. As the number of APIs continues to grow, they have become a common target for cyberattacks, making API security an important challenge. Existing API security solutions mainly rely on predefined rules and signature-based detection methods, which may not effectively identify new attack patterns or suspicious user behavior. This project proposes an Intelligent API Threat Detection framework using Graph Learning that represents API interactions as a graph of clients, users, endpoints and resources to capture the relationships between them. A Graph Attention Network (GAT) learns from this structure and from node and edge features to identify unusual API activity that may indicate a security threat. A Random Forest classifier on tabular request features serves as a baseline. Each analyzed request is assigned a threat class and a risk score between 0 and 100. Explainable Artificial Intelligence (XAI) is integrated to give clear and understandable explanations for detected anomalies, improving the transparency and interpretability of the detection process. The system is implemented as a FastAPI backend with a MySQL store and a web dashboard that shows threat overviews, risk-score distribution, threat-type distribution, the API interaction graph and model explanations. In the current prototype, the Random Forest baseline reaches an accuracy of 99.86% and the Graph Attention Network reaches 98.49%. The prototype graph contains 354 nodes and 10,866 edges. The project aims to provide an accurate, reliable and explainable approach to API threat detection while offering an efficient and scalable solution for modern API-based applications.

ACKNOWLEDGEMENT

First, we thank the almighty God for the successful completion of the project. Our sincere thanks to our Chairman **Mr. S. Meganathan B.E., F.I.E.,** for his sincere endeavor in educating us in his premier institution. We would like to express our deep gratitude to our beloved Chairperson **Dr. Thangam Meganathan Ph.D.,** for her enthusiastic motivation which inspired us a lot in completing this project and Vice Chairman **Mr. Abhay Shankar Meganathan B.E., M.S.,** for providing us with the requisite infrastructure.

We also express our sincere gratitude to our college Principal **Dr. S.N. Murugesan M.E., Ph.D.,** and **Dr. P. Tamilselvi Ph.D., Professor and Head, Department of Computer Science and Engineering (Cyber Security)** and our project guide **Mrs. Seethalakshmi T, Assistant Professor,** for her encouragement and guiding us throughout the project. We would like to thank our project coordinator **Mr. Benedict J.N., M.E., Assistant Professor (SG),** for his encouragement and guidance towards successful completion of this project and to our parents, friends, all faculty members and supporting staffs for their direct and indirect involvement in successful completion of the project for their encouragement and support.

KAVIYA J.J (2116231901019)

SASIKUMAR B. (2116231901047)

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iii**
	LIST OF TABLES	**vi**
	LIST OF FIGURES	**vii**
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	1
	1.2 PROBLEM STATEMENT	3
	1.3 OBJECTIVES	3
	1.4 SCOPE	4
	1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS	4
	1.6 SDG ALIGNMENT	5
2	**LITERATURE REVIEW**	**6**
	2.1 EXISTING SYSTEM WITH SECURITY APPROACHES	6
	2.2 RESEARCH GAPS	8
	2.3 PROPOSED SYSTEM	8
	2.3.1 OVERVIEW OF THE PROPOSED SYSTEM	8
	2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM	8
	2.3.3 ADVANTAGES OF PROPOSED SYSTEM	9
3	**SYSTEM DESIGN**	**10**
	3.1 SYSTEM ARCHITECTURE	10
	3.2 MODULE DESIGN	11
	3.3 DATA FLOW / UML DIAGRAMS	12
	3.4 THREAT MODEL AND ATTACK SURFACE	14
	3.5 SECURITY CONTROLS	15
4	**SYSTEM DESCRIPTION**	**16**
	4.1 DATA COLLECTION	16
	4.2 DATA PREPROCESSING	16
	4.2.1 ALGORITHMS / METHODOLOGY	17
	4.2.2 RESULTS AND DISCUSSION	19
5	**CONCLUSION**	**23**
	REFERENCES	**24**
	APPENDIX	**25**

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	SDG ALIGNMENT	5
2.1	LITERATURE SURVEY SUMMARY	7
3.1	BACKEND MODULES AND API ROUTERS	12
3.2	SECURITY CONTROLS AGAINST IDENTIFIED THREATS	15
4.1	TECHNOLOGY STACK	19
4.2	SAMPLE DETECTION RESULTS	20
4.3	MODEL PERFORMANCE COMPARISON	22

LIST OF FIGURES

FIGURE	TITLE	PAGE NO.
1.1	API INTERACTION GRAPH CONCEPT	2
1.2	THREAT DETECTION WORKFLOW	2
3.1	SYSTEM ARCHITECTURE	10
3.2	USE CASE DIAGRAM	13
3.3	SEQUENCE DIAGRAM	13
4.1	GAT-BASED CLASSIFICATION PIPELINE	18
4.2	DASHBOARD OVERVIEW	19
4.3	THREAT DETECTIONS TABLE	20
4.4	RISK SCORE DISTRIBUTION	21
4.5	THREAT TYPE DISTRIBUTION AND GRAPH LEARNING ROLE	21
4.6	API INTERACTION GRAPH VIEW	22

CHAPTER 1
INTRODUCTION

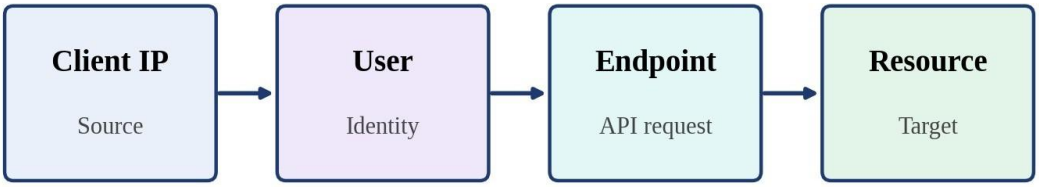
1.1 BACKGROUND

Application Programming Interfaces (APIs) allow web, mobile and cloud applications to exchange data and invoke functionality in a standard way. Banking, e-commerce, healthcare and many other digital services now depend on APIs. Because APIs expose business logic and data directly, they have become a frequent target for attackers. Common API threats include credential stuffing, broken authentication, abuse of object-level authorization, excessive data exposure, scraping and denial of service[10].

Traditional API protection relies on predefined rules, web application firewalls, rate limits and signatures of known attacks. These controls work well against known patterns, but they may struggle with new attack patterns, low-and-slow attacks, and abuse that looks legitimate when each request is seen in isolation. For example, in credential stuffing, an attacker replays leaked username and password pairs against a login endpoint. Each request is a valid-looking POST, and the malicious behavior appears only when many requests, users and failures are viewed together.

Machine learning offers an alternative in which a model learns patterns of normal and malicious behavior from labeled samples. Classifiers such as the Random Forest[7] work on tabular features extracted from each request, such as method, endpoint, status code and timing. However, tabular models treat each request independently and cannot directly use the relationships between clients, users, endpoints and resources.

Graph learning addresses this limitation. API interactions can be represented as a graph in which entities (client IP, user, endpoint, resource) are nodes and interactions are edges, as shown in Figure 1.1. Graph Neural Networks (GNNs) such as the Graph Convolutional Network[11] and the Graph Attention Network (GAT)[1] pass messages between neighboring nodes to learn representations that reflect both an entity's own features and its context. The GAT additionally learns attention weights, which indicate how much each neighbor should influence the result.



Nodes carry request and behavioural features; edges encode who accessed what

Figure 1.1 – API Interaction Graph Concept

Detection alone is not enough for a security analyst. A model that flags a request without a reason is hard to trust and hard to act on. Explainable AI (XAI) techniques therefore complement detection by showing which features or relationships led to an alert. This project combines graph-based detection, risk scoring and explainability in a single prototype with a monitoring dashboard.

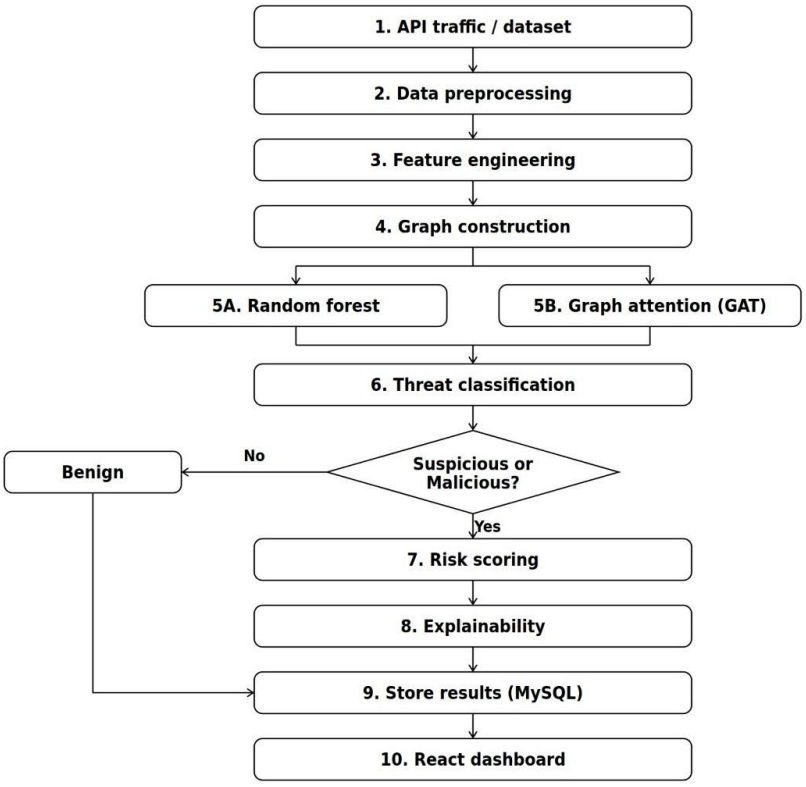


Figure 1.2 – Threat Detection Workflow

The workflow in Figure 1.2 begins with API traffic or dataset records, which are preprocessed and converted into engineered features. The features are used to construct the API interaction graph, which is analyzed by the Random Forest baseline and the Graph

Attention Network (GAT). Both models feed the threat classification step. Requests classified as benign are stored directly, while suspicious or malicious requests are assigned a risk score and an explanation before all results are stored in MySQL and presented on the React dashboard.

1.2 PROBLEM STATEMENT

APIs are now a primary attack surface, yet many deployed defenses depend on fixed rules and signatures. Such defenses may miss new attack patterns and cannot easily capture how users, requests and endpoints relate to one another. Where machine learning is used, it often treats requests independently and returns a verdict without an explanation that analysts can understand.

Therefore, the problem addressed by this project is:

> *To design and develop an explainable API threat detection system that models API interactions between clients, users, endpoints and resources as a graph, uses graph learning to detect abnormal API behavior, and presents risk scores and explanations to security analysts through a monitoring dashboard.*

1.3 OBJECTIVES

The major objectives of the project are:

1. To study existing API security and anomaly detection approaches, including machine learning and graph-based methods.
2. To preprocess API request data and extract request and behavioral features.
3. To represent API interactions as a graph of client, user, endpoint and resource nodes.
4. To implement a Random Forest model as a tabular baseline for threat detection.
5. To implement a Graph Attention Network that learns from the API interaction graph.
6. To classify API requests (for example benign or credential stuffing) and assign a risk score from 0 to 100.
7. To provide human-readable explanations for detected threats using Explainable AI.

8. To evaluate the models using accuracy and other standard classification metrics.
9. To store analyzed requests in a database and expose results through a REST API.
10. To develop a web dashboard that presents threats, risk analysis, the API graph and explanations.

1.4 SCOPE

The scope of the project includes:

- API request data preprocessing and feature extraction.
- Construction of a heterogeneous API interaction graph.
- Random Forest baseline classification.
- Graph Attention Network based threat detection.
- Risk scoring and threat classification of API requests.
- Explainability of model predictions.
- FastAPI backend with MySQL storage.
- Web dashboard for monitoring and analysis.
- Comparison of the tabular baseline and the graph-based model.

The system is a research prototype for detecting and explaining suspicious API behavior. It is not intended to replace production API gateways, web application firewalls or enterprise security monitoring products. In its current form, the system analyzes recorded or simulated API requests, and it does not block live traffic by itself. Automatic blocking and rate limiting are listed as future work.

1.5 SECURITY REQUIREMENTS / SECURITY CONSIDERATIONS

Since the system processes API traffic data and exposes a dashboard, security is an important design consideration.

1. **Input validation** – Data received by the backend should be validated before processing.
2. **Restricted cross-origin access** – The backend should accept browser requests only from the trusted dashboard origin.
3. **Credential protection** – Database credentials should be kept in configuration rather than in source code.

4. **Data confidentiality** – Request logs may contain sensitive information; they should be stored in the database with access control.
5. **Model protection** – Trained models and the graph should be stored separately from user-facing components and protected from modification.
6. **Controlled error handling** – Error messages should not expose internal details to users.
7. **Availability** – Limits on request volume and size should be applied to avoid resource exhaustion.
8. **Authentication and authorization** – Access to the dashboard and API should be restricted to authorized analysts where deployed.
9. **Dataset security** – Only trusted and established datasets should be used during development.

1.6 SDG ALIGNMENT

The project can be aligned primarily with the following Sustainable Development Goals:

Table 1.1 – SDG Alignment

S. No.	SDG No. & Goal	Relevant SDG Target	Project Contribution / Alignment
1	SDG 9 – Industry, Innovation and Infrastructure	Innovation and resilient digital infrastructure	Develops an AI-based approach to protect API infrastructure from abuse and attacks.
2	SDG 16 – Peace, Justice and Strong Institutions	Strengthening secure and accountable institutions	Supports cybersecurity through automated and explainable detection of API threats.
3	SDG 8 – Decent Work and Economic Growth	Secure and productive digital environments	Helps keep API-based digital services safe and reliable for businesses and users.

CHAPTER 2
LITERATURE REVIEW

2.1 EXISTING SYSTEM WITH SECURITY APPROACHES

Rule-Based and Signature-Based API Protection

API gateways and web application firewalls inspect requests against predefined rules and signatures of known attacks.

API request → Match against rules / signatures → Match? → Block / Allow

The major limitation is that new attack patterns and subtle behavioral abuse may not match any rule, and rules need constant manual maintenance.

Rate Limiting and Threshold-Based Monitoring

Simple thresholds, such as a maximum number of failed logins per minute, can slow down some attacks. Attackers can stay below thresholds by spreading requests across many IP addresses or users, and thresholds do not explain the wider context of an attack.

Machine Learning-Based Detection

Machine learning approaches extract features from API logs and train classifiers to identify abnormal behavior. Alam et al.[2] applied machine learning to open banking API access logs, and Sultana et al.[5] studied supervised and unsupervised models on large enterprise API logs.

API logs → Feature extraction → Machine learning model → Normal / Malicious

These models can reach high accuracy, but they typically treat each request or session as an independent record.

Graph-Based Detection

Graph-based methods model entities and their interactions. Zhang et al.[4] used Graph Convolutional Networks to model inter-service communication in microservices, and Pirapuraj et al.[3] used graph neural networks and clustering on API flow graphs to detect API misuse. Li et al.[6] built graph models from OpenAPI specifications and provided visual explanations of suspicious behavior.

Interactions → Graph → GNN → Node / edge representation → Classification

Literature Survey Summary

Table 2.1 summarizes the five studies that guided this project.

Table 2.1 – Literature Survey Summary

No.	Paper (Authors, Year)	Methodology / Focus	Key Findings and Relevance
1	**Open Banking API Security: Anomalous Access Behaviour** Alam et al., 2023	ML-based anomaly detection using API access logs and behavioral features in an open banking environment.	Identified anomalous access patterns in real banking APIs with high detection accuracy; highlighted the value of behavioral analysis. Supports ML for API anomaly detection.
2	**AFGNN: API Misuse Detection using Graph Neural Networks and Clustering** Pirapuraj et al., 2026	API Flow Graph representation with GNNs (GCN, GAT, RGCN) and BIRCH clustering for API misuse in Java source code.	Graph captures call sequence, data flow and control flow; outperformed existing baselines. Validates graph learning and inspires the user–request–endpoint graph model.
3	**Anomalous Distributed Traffic: Detecting Cyber Attacks amongst Microservices using GCNs** Zhang et al., 2022	GCN models inter-service communication as a graph to detect anomalous traffic in microservices.	GCNs detected attacks in distributed systems and outperformed traditional ML. Supports modeling users, requests and endpoints as a graph.
4	**API Security in Large Enterprises: Leveraging Machine Learning for Anomaly Detection** Sultana et al., 2021	Supervised and unsupervised ML on large-scale enterprise API logs.	Identified features indicative of malicious API activity and the practical challenges of deploying ML for API security. Justifies ML in this project.
5	**API Security Based on Automatic OpenAPI Mapping** Li et al., 2026	Graph representation of API structure and behavior from OpenAPI specifications, with anomaly detection and visualization.	Detected abnormal and malformed requests and gave explainable visualization of suspicious behavior. Highly relevant to graph-based modeling and the XAI component.

Key Insights from the Literature

- Machine learning and graph learning are effective for detecting anomalous behavior in APIs and network systems.
- Graph representations capture complex relationships and behavioral patterns that independent records cannot.
- Real-world studies show the importance of behavioral analysis for API security.
- Explainability is increasingly important for practical deployment and analyst trust.

2.2 RESEARCH GAPS

Based on the literature, the following gaps were identified:

1. Existing work targets specific domains such as banking, enterprise logs, source code or network traffic.
2. Studies typically model either API structure or traffic, but not the combined relationship between users, requests and API endpoints.
3. Integration of graph learning with explainable AI for API threat detection is limited.
4. Few studies provide a unified framework that detects anomalous API behavior and also offers clear, interpretable explanations and a monitoring interface for security analysts.

The project therefore addresses the following research question:

> *Can graph learning over user–request–endpoint relationships detect API threats accurately, while explainable outputs help analysts understand and trust the results?*

2.3 PROPOSED SYSTEM

2.3.1 OVERVIEW OF THE PROPOSED SYSTEM

The proposed system consists of the following stages:

API Data → Cleaning and Normalization → Sessionization → Feature Extraction → Graph Construction → Graph Attention Network / Random Forest → Anomaly Score and Threat Classification → Explainability → Alerts, Dashboard and Reports

The Random Forest model provides a tabular baseline, and the Graph Attention Network learns from the API interaction graph. Predictions are converted to a risk score and a threat class, stored in MySQL, and presented on a web dashboard together with explanations.

2.3.2 OBJECTIVES OF THE PROPOSED SYSTEM

- Detect abnormal and malicious API behavior automatically.
- Represent API interactions as a graph of clients, users, endpoints and resources.
- Apply a Graph Attention Network to learn relationship-aware behavior.
- Assign risk scores and threat classes to individual requests.

- Provide understandable explanations for flagged activity.
- Offer a dashboard with statistics, risk analysis and graph visualization.
- Compare the graph-based model with a tabular baseline.

2.3.3 ADVANTAGES OF PROPOSED SYSTEM

1. **Relationship-aware detection** – The graph captures how clients, users and endpoints interact, which independent request records cannot show.
2. **Attention mechanism** – The GAT learns which neighboring information matters more during message passing.
3. **Risk scoring** – A 0–100 risk score allows graded severity instead of only a yes or no label.
4. **Explainability** – Analysts can see why a request was flagged, improving trust and speed of response.
5. **Baseline comparison** – The Random Forest model gives a reference to measure the benefit of graph learning.
6. **Practical deployment** – A FastAPI service, a MySQL store and a React-based dashboard make the system easy to demonstrate and extend.

CHAPTER 3
SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

Figure 3.1 shows the layered architecture of the system. It has five main blocks, from the API data source to the web dashboard, together with a MySQL database that stores the results.

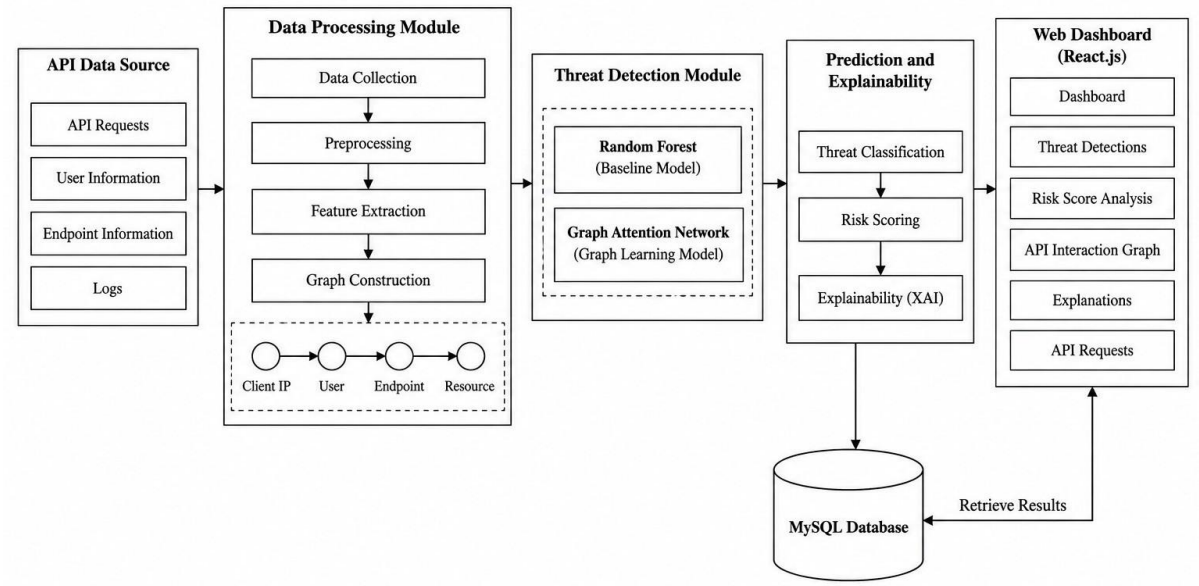


Figure 3.1 – System Architecture

- **API data source** – API requests, user information, endpoint information and logs.
- **Data processing module** – Data collection, preprocessing, feature extraction and graph construction. The graph links Client IP, User, Endpoint and Resource nodes.
- **Threat detection module** – The Random Forest baseline model and the Graph Attention Network (graph learning model).
- **Prediction and explainability** – Threat classification, risk scoring and Explainable AI (XAI) explanations of why activity was flagged.
- **MySQL database** – Stores the classified requests, risk scores and explanations; the dashboard retrieves results from it.
- **Web dashboard (React.js)** – Dashboard, threat detections, risk score analysis, API interaction graph, explanations and API requests.

The current prototype implements the data processing, graph construction, Random Forest baseline, GAT detection, risk scoring, dashboard and explanation views. Temporal modeling and automated alert delivery are planned extensions.

3.2 MODULE DESIGN

Module 1 — Data Management and Preprocessing

Responsible for loading API request records, removing duplicates, handling missing values, encoding categorical fields such as method, endpoint and status, and scaling numerical features. It also prepares labeled data and the train, validation and test split.

Module 2 — Sessionization and Feature Extraction

Groups requests into sessions by user and time window and extracts behavioral features from requests and users, for example request counts, failed authentication counts and endpoint diversity.

Module 3 — Graph Construction

Builds the API interaction graph (Client IP → User → Endpoint → Resource). Each entity is a node with a feature vector, and each interaction is an edge. In the current prototype, the graph has 354 nodes and 10,866 edges and is exposed through the graph service.

Module 4 — Random Forest Baseline

A Random Forest classifier[7] trained on tabular request features. It produces the class prediction and risk score shown in the Threat Detections view, where the model column reads Random Forest.

Module 5 — Graph Attention Network

The GAT[1] receives the node feature matrix and edge index. Multi-head attention layers aggregate information from neighboring nodes, and a classification layer outputs the class probabilities.

Module 6 — Risk Scoring and Threat Classification

Converts model outputs to a risk score from 0 to 100 and assigns a level (Low, Suspicious or High) and a detected scenario such as credential stuffing or benign.

Module 7 — Explainability

Produces human-readable explanations for flagged requests, highlighting influential features and relationships, and displays them in the Explainability view.

Module 8 — Backend Service and Database

A FastAPI application exposes the detection results through REST routers and stores analyzed requests in MySQL. Table 3.1 lists the backend modules.

Table 3.1 – Backend Modules and API Routers

Module	Responsibility
main.py	Creates the FastAPI application, enables CORS for the dashboard origin, registers all routers and provides the /health endpoint, which reports service and MySQL connection status.
database.py	Creates MySQL connections from configured host, port, user, password and database name; includes a connection test used by the health check.
threats router	Returns analyzed API requests with scenario, method, endpoint, status, detection result, risk score and model.
stats router	Returns summary statistics: total requests, threats, benign requests, average risk and threat-type counts.
explain router	Returns explanations for model predictions.
simulate router	Generates simulated API request scenarios for analysis and demonstration.
gat router and gat_service	Loads the trained GAT and graph and serves graph-based predictions.
graph router (/api/graph/data)	Returns the graph as lists of nodes (id and features) and edges (source and target) with node and edge counts.

Module 9 — Web Dashboard

A browser dashboard with five views: Dashboard, Threats, API Graph, Explainability and API Requests. It calls the backend and presents statistics, charts, tables and the graph overview.

3.3 DATA FLOW / UML DIAGRAMS

The data flow of the system is shown in Figure 1.2. The use case and sequence diagrams below describe how an analyst interacts with the system.

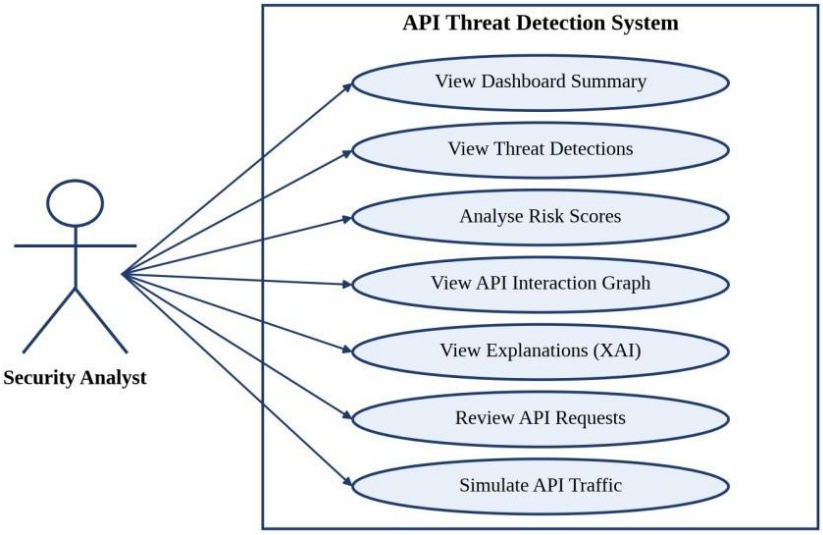


Figure 3.2 – Use Case Diagram

The security analyst is the primary actor. The analyst can view the dashboard summary, threat detections, risk scores, the API interaction graph and explanations, review stored API requests, and run simulated traffic.

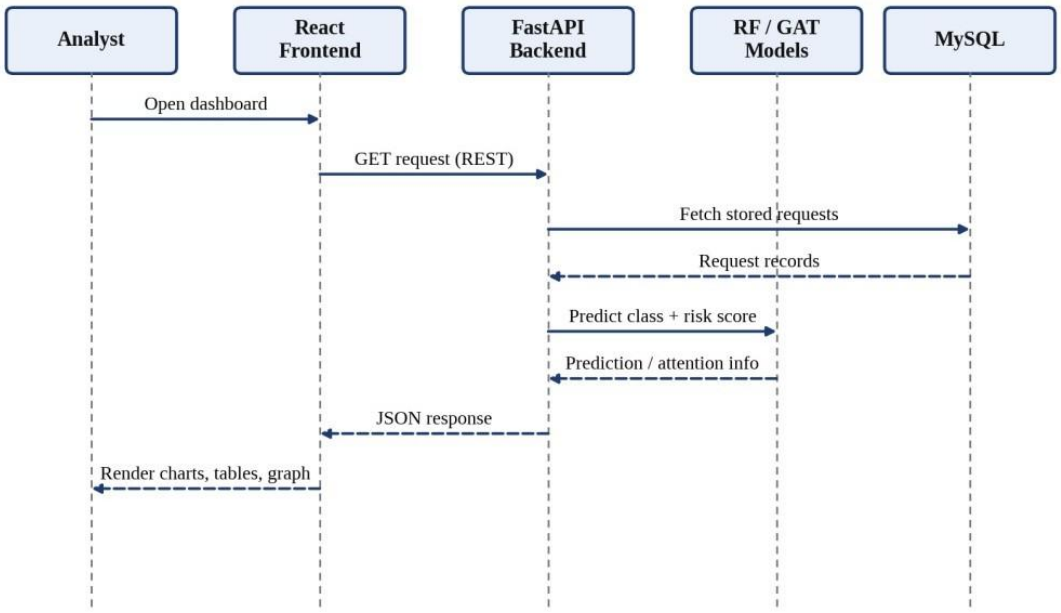


Figure 3.3 – Sequence Diagram

When the analyst opens the dashboard, the React frontend sends REST requests to the FastAPI backend. The backend reads stored records from MySQL, obtains predictions and risk scores from the Random Forest and GAT models, and returns a JSON response, which the frontend renders as charts, tables and graph views.

3.4 THREAT MODEL AND ATTACK SURFACE

Since the system exposes an API and a dashboard and relies on a trained model, the main attack surfaces include:

1. **Malicious or malformed API input** – Specially crafted requests may attempt to exploit parsing or database access.
2. **Credential stuffing and brute force** – The very threats the system detects may also be used to probe the system's own login endpoints.
3. **Evasion of the model** – An attacker may shape traffic to look benign, for example by spreading requests across many IPs or users.
4. **Training data poisoning** – Tampered training data or labels could weaken the detector.
5. **Model manipulation** – Unauthorized modification of the trained model or graph could change predictions.
6. **Web dashboard attacks** – The dashboard and backend may face cross-origin and injection attacks if not protected.
7. **Database exposure** – Stored request logs may contain sensitive information.
8. **Denial of service** – Repeated expensive requests, such as retrieving the full graph, could consume resources.

3.5 SECURITY CONTROLS

The following controls are incorporated or planned:

Table 3.2 – Security Controls Against Identified Threats

Threat	Security Control
Malformed input	Input validation of request data before processing
Cross-origin abuse	CORS restricted to the dashboard origin (http://localhost:5173 in development)
Credential exposure	Database credentials loaded from configuration, not hardcoded
Model modification	Restricted file access to trained model and graph artifacts
Data poisoning	Use of trusted datasets and review of labels
Evasion	Graph context and periodic retraining with analyst feedback
Denial of service	Request and payload limits; pagination of large responses (planned)
Unauthorized access	Authentication and role-based access for the dashboard (planned)
Information leakage	Controlled error messages that avoid internal details (planned improvement)
Data confidentiality	Access-controlled MySQL database for stored requests

CHAPTER 4
SYSTEM DESCRIPTION

4.1 DATA COLLECTION

The system is developed and evaluated with API security data containing both benign requests and attack scenarios. Each record describes one API request with fields such as the HTTP method, endpoint, status code, client IP, user, timestamp and a scenario label. The scenarios seen in the prototype include benign traffic and credential stuffing, for example repeated POST requests to /api/v1/auth/login that return status 401.

The dataset used for the final experiments is [dataset name and source – to be specified], and the exact number of records and classes used should be reported from the actual dataset. The data are divided into training, validation and test sets in the ratio [actual split].

Dataset considerations

The following issues should be considered:

- Class imbalance between benign and attack records
- Duplicate or highly similar samples
- Data leakage between training and test sets, especially when graph neighbors are shared
- Distribution of attack scenarios
- Training/test separation

The test set should contain samples that were not used during model training. The very high accuracy of the baseline model should be confirmed with a leakage-free split and, where possible, on traffic from a different source.

4.2 DATA PREPROCESSING

The preprocessing and graph construction pipeline is:

API Records → Cleaning → Encoding and Scaling → Sessionization → Feature Extraction → Graph Construction (Nodes, Edges) → Model Input

Cleaning removes duplicate records and handles missing values. Categorical features such as method, endpoint and status are encoded, and numerical features are scaled. Requests

are grouped into sessions by user and time window, and behavioral features are extracted for requests and users. The resulting entities and interactions form the API interaction graph used by the GAT.

4.2.1 ALGORITHMS / METHODOLOGY

Algorithm 1 — API Interaction Graph Construction

Input: Preprocessed API request records

1. Load the cleaned records.
2. Identify the entities in each record: client IP, user, endpoint and resource.
3. Create one node for each unique entity and attach its feature vector.
4. For each request, add edges that connect Client IP → User → Endpoint → Resource.
5. Assemble the node feature matrix X and the edge index.
6. Store the graph for the GAT and visualization. The prototype graph has 354 nodes and 10,866 edges.

Algorithm 2 — Random Forest Classification

Input: Tabular features of an API request

1. Pass the feature vector to every decision tree in the forest.
2. Each tree votes for a class.
3. Select the class with the majority of votes.
4. Derive a risk score between 0 and 100 from the model output.
5. Return the predicted class and risk score.

Algorithm 3 — Graph Attention Network Classification

Input: Graph with node features X and edge index

1. For each node i and neighbor j, compute an attention score from the transformed features of both nodes.
2. Normalize the scores over the neighbors of node i to obtain attention weights.
3. Aggregate the weighted neighbor features to update node i; repeat with several attention heads.
4. Stack GAT layers to obtain node embeddings.

5. Pass the embeddings through a classification layer and apply softmax.
6. Return the predicted class and its probability.

$$e_{ij} = \text{LeakyReLU}(a^{T} [W h_i \Vert W h_j])$$

$$\alpha_{ij} = \text{softmax}_j (e_{ij})$$

$$h_i' = \sigma (\Sigma_{j \in N(i)} \alpha_{ij} \cdot W h_j)$$

Here h_i is the feature vector of node i, W is a learnable weight matrix, a is the attention vector, α_{ij} is the attention weight of neighbor j for node i, and N(i) is the set of neighbors of i[1].

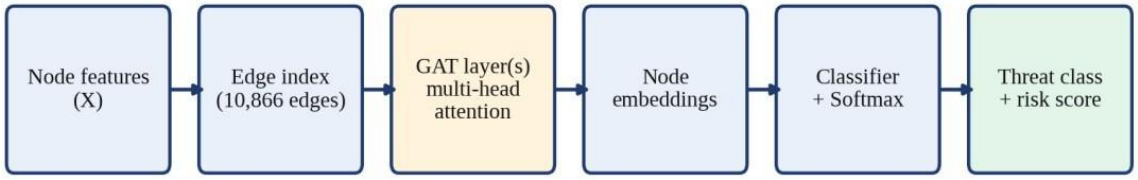


Figure 4.1 – GAT-Based Classification Pipeline

Algorithm 4 — Risk Scoring, Explanation and Alerting

Input: Model prediction for an API request

1. Convert the prediction to a risk score from 0 to 100.
2. Assign a level: Low, Suspicious or High.
3. Identify the scenario (for example credential_stuffing or benign).
4. Generate an explanation highlighting influential features.
5. Store the request, score, class and explanation in MySQL.
6. Display the result in the dashboard and raise an alert for high-risk requests.

Training Methodology

Dataset → Preprocessing → Feature extraction → Graph construction → Train / Validation / Test split → Model training (Random Forest, GAT) → Validation → Best model → Test evaluation → Deployment through FastAPI

4.2.2 RESULTS AND DISCUSSION

Implementation

The system is implemented as a FastAPI backend with a MySQL database and a web dashboard. Table 4.1 lists the main technologies.

Table 4.1 – Technology Stack

Component	Technology
Backend	Python, FastAPI (title “Intelligent API Threat Detection”, version 1.0.0)
Database	MySQL, accessed through mysql-connector
Baseline model	Random Forest classifier on tabular features
Graph model	Graph Attention Network on a graph with node features and edge index
Frontend	Browser dashboard (React), served at localhost:5173 during development
API style	REST with JSON; routers for threats, stats, explain, simulate, gat and graph

Dashboard Overview

Figure 4.2 shows the dashboard overview. In the test run, 4 API requests were analyzed: 3 were detected as threats and 1 was benign, with an average risk of 86. The Model Performance panel reports 99.86% for the Random Forest (tabular baseline) and 98.49% for the Graph Attention Network (graph-based detection).

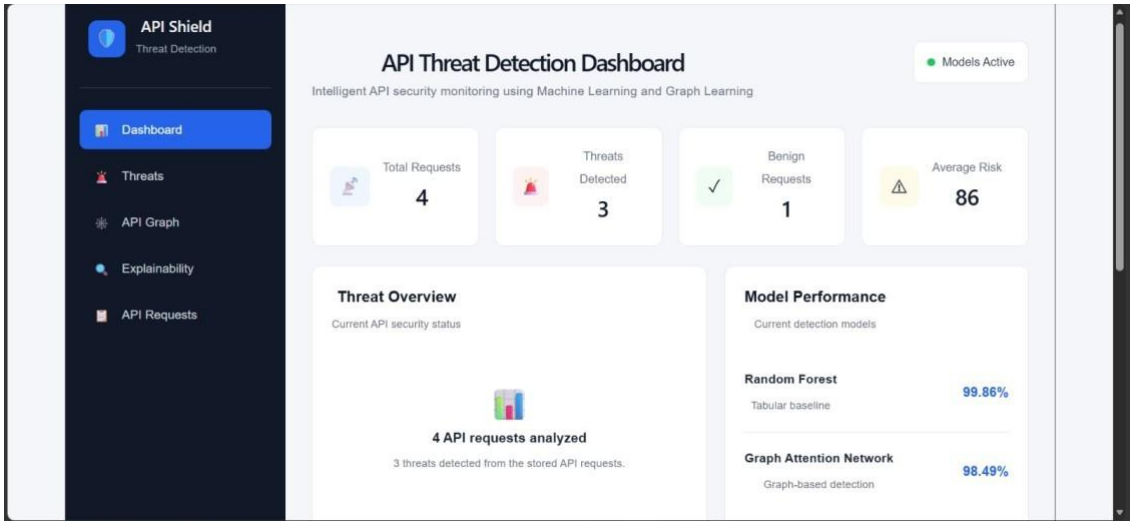


Figure 4.2 – Dashboard Overview

Threat Detections

The Threats view lists each analyzed request with its scenario, method, endpoint, status, detection result, risk score and model. Table 4.2 gives the four records of the test run.

Table 4.2 – Sample Detection Results

No.	Scenario	Method	Endpoint	Status	Detection	Risk
1	credential_stuffing	POST	/api/v1/auth/login	401	Threat	100
2	benign	GET	/api/v1/products	200	Benign	44
3	credential_stuffing	POST	/api/v1/auth/login	401	Threat	100
4	credential_stuffing	POST	/api/v1/auth/login	401	Threat	100

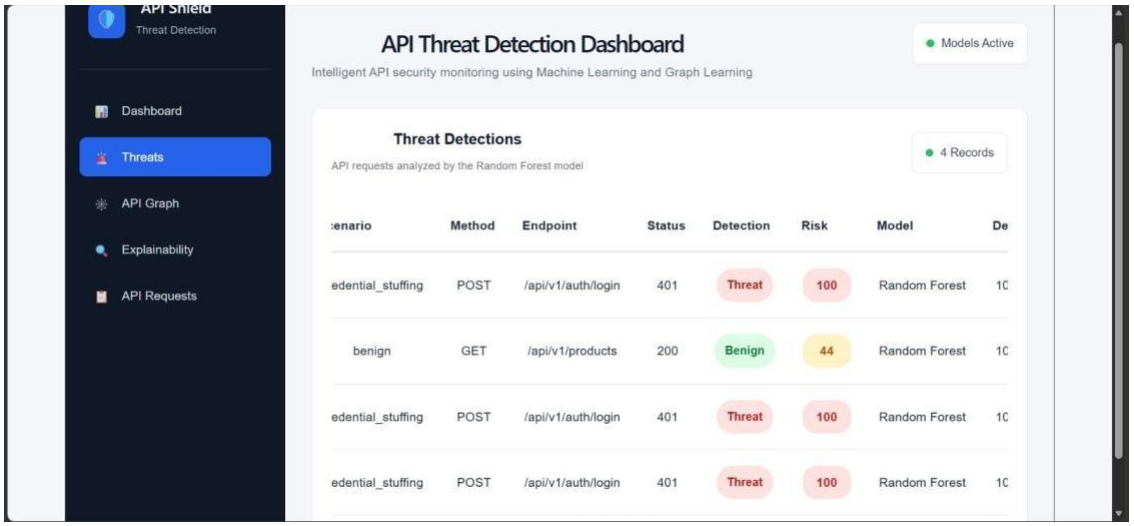


Figure 4.3 – Threat Detections Table

The three credential stuffing requests, repeated POST calls to the login endpoint that fail with status 401, receive the maximum risk score of 100. The benign GET request to /api/v1/products succeeds with status 200 and receives a lower score of 44.

Risk Score Analysis

Figure 4.4 plots the risk score for each request in sequence against Low, Suspicious and High bands. Three requests fall in the High band and one in the Suspicious band. The summary cards show 4 total requests, 3 high-risk, 1 suspicious and an average risk of 86. The benign request with score 44 is classified as benign but sits in the Suspicious band, which shows that the score gives graded severity beyond the binary label and can be used to prioritize analyst attention.

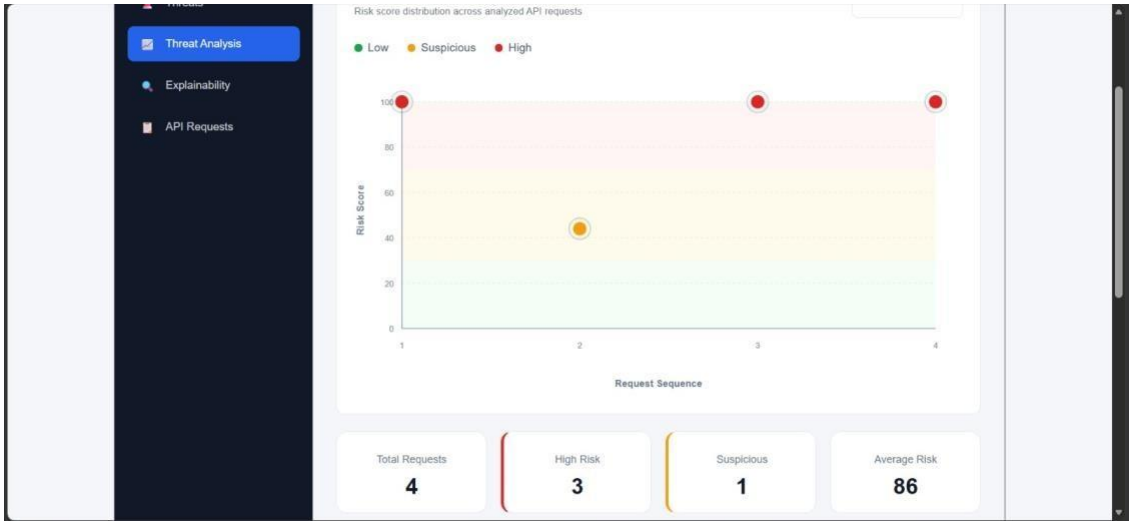


Figure 4.4 – Risk Score Distribution

Threat Type Distribution and Role of Graph Learning

The threat type distribution shows 3 credential_stuffing requests and 1 benign request. The dashboard also explains the role of the Graph Attention Network in four steps: graph construction, node and edge features, graph attention, and threat prediction.

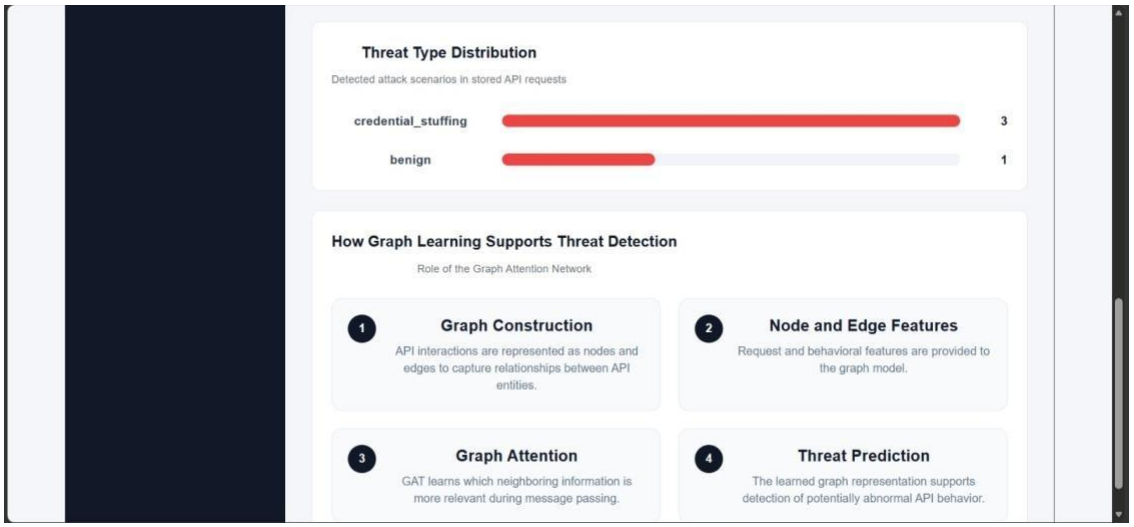


Figure 4.5 – Threat Type Distribution and Graph Learning Role

API Interaction Graph

The API Graph view shows the relationship chain Client IP (source) → User (identity) → Endpoint (API request) → Resource (target). The loaded graph contains 354 nodes and 10,866 edges.

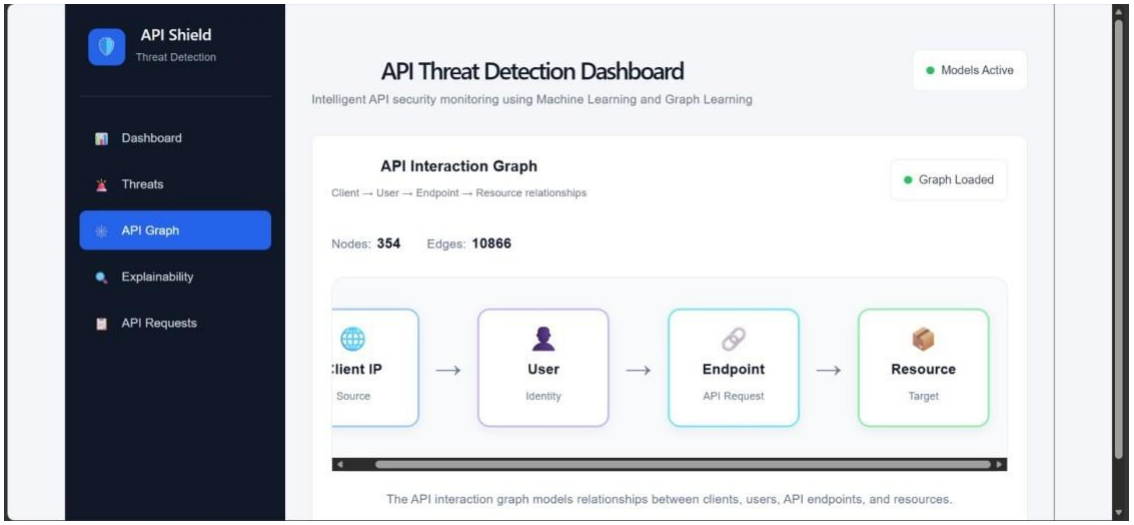


Figure 4.6 – API Interaction Graph View

Model Performance Comparison

Table 4.3 compares the two models. Accuracy values are those reported by the system; the remaining metrics are to be filled in from the evaluation on the test set.

Table 4.3 – Model Performance Comparison

Metric	Random Forest (tabular baseline)	Graph Attention Network
Accuracy	99.86%	98.49%
Precision	[___]	[___]
Recall	[___]	[___]
F1-score	[___]	[___]
Inference time	[___ms]	[___ms]

The Random Forest baseline achieves slightly higher accuracy (99.86%) than the GAT (98.49%) on the evaluated data. This is common when strong tabular signals, such as repeated failed logins, already separate the classes. The value of the GAT lies elsewhere: it uses relationships between clients, users and endpoints, which can reveal coordinated or distributed behavior that single-request features may not show. The two models are therefore complementary, and a more difficult dataset with distributed attacks is needed to measure the benefit of graph context fully.

CHAPTER 5
CONCLUSION

This project presents an Intelligent API Threat Detection system that uses graph learning to detect abnormal API behavior. API interactions between clients, users, endpoints and resources are modeled as a graph, a Graph Attention Network learns from this structure, and a Random Forest provides a tabular baseline. Each request receives a threat class and a risk score, and explainability views help analysts understand the results.

The prototype was implemented with a FastAPI backend, a MySQL database and a web dashboard with views for threats, the API graph, explainability and API requests. The Random Forest baseline reached an accuracy of 99.86% and the Graph Attention Network reached 98.49%. In the test run, the system flagged all three credential stuffing requests with a risk score of 100 and gave a lower score of 44 to a benign product request. The prototype graph contains 354 nodes and 10,866 edges.

The current work has some limitations. The sample run contains only four requests, the reported accuracies depend on the dataset and split used, and temporal modeling, authentication and automated response are not yet implemented.

The project can be extended in several directions: evaluating additional attack types such as broken object-level authorization, scraping and denial of service; adding temporal models (RNN, GRU or LSTM); comparing GraphSAGE and GIN with the GAT; adding GNNExplainer or SHAP explanations; processing live API traffic with automated blocking or rate limiting; and testing on larger and more diverse API security datasets.

REFERENCES

1. P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in Proc. Int. Conf. Learning Representations (ICLR), 2018.
2. Alam et al., “Open Banking API Security: Anomalous Access Behaviour,” 2023.
3. Pirapuraj et al., “AFGNN: API Misuse Detection using Graph Neural Networks and Clustering,” 2026.
4. Zhang et al., “Anomalous Distributed Traffic: Detecting Cyber Security Attacks amongst Microservices using Graph Convolutional Networks,” 2022.
5. Sultana et al., “API Security in Large Enterprises: Leveraging Machine Learning for Anomaly Detection,” 2021.
6. Li et al., “API Security Based on Automatic OpenAPI Mapping,” 2026.
7. L. Breiman, “Random forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
8. S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in Advances in Neural Information Processing Systems (NeurIPS), 2017.
9. R. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, “GNNExplainer: Generating explanations for graph neural networks,” in Advances in Neural Information Processing Systems (NeurIPS), 2019.
10. OWASP Foundation, “OWASP API Security Top 10,” 2023.
11. T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in Proc. Int. Conf. Learning Representations (ICLR), 2017.

APPENDIX

Appendix A — Project Code

Backend (FastAPI):

- main.py – application setup, CORS, router registration, health check
- database.py – MySQL connection and connection test
- config – database settings
- api/threats.py, api/stats.py, api/explain.py, api/simulate.py, api/gat.py, api/graph.py – routers
- services/gat_service.py – GAT model and graph loading
- Frontend dashboard – React application (Dashboard, Threats, API Graph, Explainability, API Requests)

Excerpt: main.py

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.database import test_connection
from app.api import threats, stats, explain, simulate, gat, graph

app = FastAPI(
    title="Intelligent API Threat Detection",
    description="API Threat Detection and Risk Assessment Backend",
    version="1.0.0"
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(threats.router)
app.include_router(stats.router)
app.include_router(explain.router)
app.include_router(simulate.router)
app.include_router(gat.router)
app.include_router(graph.router)

@app.get("/health")
def health_check():
    mysql_status = test_connection()
    return {
        "status": "healthy",
        "service": "API Threat Detection Backend",
        "mysql": "connected" if mysql_status else "disconnected"
    }
```

Appendix B — Dataset Classes

Scenario classes observed in the prototype: benign, credential_stuffing.

[List all attack scenarios and classes actually used in the final dataset]